

Estruturas de Repetição em Java

Trilha Java Básico

Aula de 17 de agosto de 2026

Sumário

1	Introdução	2
2	Classificação das estruturas de repetição	2
3	Comando <code>for</code>	2
3.1	Sintaxe	2
3.2	Exemplo 1 — Contador de carneirinhos	3
3.2.1	Simulação passo a passo (depuração)	3
3.3	Exemplo 2 — <code>for</code> percorrendo um array	4
3.4	<code>for each</code> (“for it”)	4
4	Comandos <code>break</code> e <code>continue</code>	5
5	Comando <code>while</code>	6
5.1	Sintaxe	6
5.2	Exemplo 3 — A mesada do Joãozinho	6
6	Comando <code>do while</code>	8
6.1	Sintaxe	8
6.2	Exemplo 4 — O telefone tocando	8
7	Comparação: <code>while</code> <i>vs</i> <code>do while</code>	9
8	Resumo e dicas de prática	9
9	Exercícios de fixação	10
9.1	Exercício 1 — <code>while</code> com condição inicial falsa	10
9.2	Exercício 2 — <code>length</code> de um array	10
9.3	Exercício 3 — Classificação das estruturas de repetição	11
9.4	Exercício 4 — <code>for</code> como acumulador	11
9.5	Exercício 5 — Loop infinito no <code>do while</code>	12

1 Introdução

Estruturas de repetição — também chamadas de *laços de repetição*, *laços de iteração* ou simplesmente *loops* — são comandos que permitem a **iteração de código**, ou seja, que os comandos presentes em um bloco sejam repetidos diversas vezes.

Por meio do *controle de fluxo de repetição*, as instruções podem acontecer recorrentemente (mais de uma vez) até que uma **condição** determine que não é mais necessária a continuação da execução daquele bloco.

Observação

Os trechos de código abaixo foram reconstruídos a partir da narração da aula. Valores e nomes exibidos na tela (ex.: elementos de um *array*) podem ter pequenas variações em relação ao código original do professor, mas a lógica e a estrutura são fiéis ao que foi explicado.

2 Classificação das estruturas de repetição

Existem três estruturas de repetição em Java:

- **for** — tradução: “para”;
- **while** — tradução: “enquanto”;
- **do while** — tradução: “faça enquanto”.

Cada uma tem sua aplicabilidade, suas particularidades estruturais e dicas de uso, detalhadas a seguir.

3 Comando for

O comando **for** (“para”) permite que uma **variável contadora** seja testada e incrementada a cada iteração. A cada execução do fluxo, essa variável pode sofrer uma alteração de valor. O **for** recebe três informações, definidas na própria chamada do comando:

1. a **variável contadora** (inicialização);
2. a **condição** (expressão booleana de validação);
3. o **incremento** (atualização da variável).

3.1 Sintaxe

```
1 for (inicializacao; expressaoBooleana; incremento) {  
2     // corpo: comandos a serem repetidos  
3 }
```

Listing 1: Estrutura do for

As três partes ficam em uma única linha, separadas por ponto e vírgula (;):

- **Inicialização** — declara a variável contadora (ex.: `int i = 0`);
- **Expressão booleana** — validação; enquanto for **verdadeira**, o corpo é executado;
- **Incremento** — atualiza a variável a cada iteração (ex.: `i++`).

O corpo (bloco entre chaves) é executado inúmeras vezes, enquanto a expressão de validação for verdadeira. O comando deixa de executar quando a condição se torna **falsa**.

O fluxo é: *inicializa* → *testa a condição* → *executa o corpo* → *incrementa* → *testa de novo* ... até a condição se tornar falsa.

3.2 Exemplo 1 — Contador de carneirinhos

Enunciado do Exemplo

Joãozinho precisa contar até 20 carneirinhos para pegar no sono. Monte um laço **for** com uma variável inteira (**int**) chamada **carneirinhos**, que inicia em 1 e, **enquanto for menor ou igual a 20**, é incrementada de 1 em 1 a cada iteração. A cada repetição, exiba a contagem atual. Ao final, depois que a condição deixar de ser verdadeira, exiba a mensagem “*Joãozinho dormiu!*”.

Solução

```

1 public class ExemploFor {
2     public static void main(String[] args) {
3         for (int carneirinhos = 1; carneirinhos <= 20; carneirinhos
4             ++){
5             System.out.println("Contando carneirinhos: " +
6                 carneirinhos);
7         }
8         System.out.println("Joãozinho dormiu!");
9     }
10 }
```

Listing 2: ExemploFor.java

3.2.1 Simulação passo a passo (depuração)

Acompanhando a execução (por exemplo, com *breakpoint* na IDE):

1. **Inicialização:** `carneirinhos = 1`.
2. **Teste:** `1 <= 20` é *verdadeiro* → executa o corpo.
3. **Corpo:** imprime “*Contando carneirinhos: 1*”.
4. **Incremento:** `carneirinhos++` → `carneirinhos = 2`.
5. **Teste:** `2 <= 20` é *verdadeiro* → imprime 2.
6. ... e assim sucessivamente até imprimir 20.
7. Após imprimir 20, o incremento leva a `carneirinhos = 21`.

8. **Teste:** $21 \leq 20$ é *falso* $\rightarrow$ o laço é encerrado.
9. O programa segue normalmente para `System.out.println("Joãozinho dormiu!")`.

Observação

A saída imprime de 1 a 20 e, ao final, “*Joãozinho dormiu!*”. Isso confirma que o programa **continua executando** normalmente após o término do laço.

3.3 Exemplo 2 — for percorrendo um array

Enunciado do Exemplo

Crie um *array* de `String` com os nomes dos alunos e percorra-o com um laço **for**, imprimindo o aluno de cada índice. Use a propriedade `length` (tamanho) do *array* como limite da condição.

Solução

```
1 public class ExemploForArray {
2     public static void main(String[] args) {
3         String[] alunos = { "Felipe", "Jonas", "Julia", "Marcos" };
4
5         for (int x = 0; x < alunos.length; x++) {
6             System.out.println("O aluno no índice " + x + " é " +
7                 alunos[x]);
8         }
9     }
```

Listing 3: ExemploForArray.java

Explicação linha a linha:

- `String[] alunos = ...` — um *array* é um conjunto de elementos do mesmo tipo. Pode ser de `String`, `int`, `double` ou de qualquer classe (clientes, funcionários etc.). Cada elemento fica entre chaves, separado por vírgula.
- `for (int x = 0; x < alunos.length; x++)` — `x` representa o **índice** do elemento. O laço continua enquanto `x` for menor que o tamanho do *array*.
- `alunos[x]` — acessa o aluno que está na posição (índice) `x` da lista.

Observação

Em *arrays*, o índice dos elementos **inicia em 0** — por isso a variável `x` começa com 0. Se `alunos.length` vale 4, os índices válidos são 0, 1, 2 e 3.

3.4 for each (“for it”)

Para interagir com *arrays* e coleções de forma mais agradável, existe o **for each**, que percorre diretamente os elementos — sem a necessidade de controlar o índice manualmente:

```

1 String[] alunos = { "Felipe", "Jonas", "Julia", "Marcos" };
2
3 for (String aluno : alunos) {
4     System.out.println("O nome do aluno é: " + aluno);
5 }

```

Listing 4: Exemplo de for each

O uso do `for each` está fortemente relacionado a cenários com *arrays*/coleções. A cada iteração, a variável temporária `aluno` (do tipo do elemento, aqui `String`) recebe **automaticamente** o elemento atual da coleção. Os dois pontos (`:`) indicam “a cada elemento de”.

4 Comandos `break` e `continue`

Dentro de laços, duas palavras têm interação direta com o contexto de repetição:

- `break` — significa **quebrar / parar / interromper**. Interrompe **todo** o laço.
- `continue` — como o nome diz, **continua o laço**. Interrompe somente a **iteração atual** (pula para a próxima).

Enunciado do Exemplo

Considere um laço `for` que conta de 1 a 5. Dentro dele existe uma condição: *se o número for igual a 3, interromper o fluxo*. Qual o resultado da execução? Em seguida, troque o `break` por `continue` e observe o que muda no resultado.

Solução

Com `break`, imprime apenas **1 e 2**: ao chegar em 3, o laço é interrompido por completo — mesmo a condição determinando a contagem de 1 a 5.

Com `continue`, imprime **1, 2, 4 e 5**: ao chegar em 3, apenas aquela iteração é pulada; o laço segue normalmente para 4 e 5.

```

1 for (int numero = 1; numero <= 5; numero++) {
2     if (numero == 3)
3         break;
4     System.out.println(numero);
5 }
6 // Saída: 1, 2

```

Listing 5: Com `break`

```

1 for (int numero = 1; numero <= 5; numero++) {
2     if (numero == 3)
3         continue;
4     System.out.println(numero);
5 }
6 // Saída: 1, 2, 4, 5

```

Listing 6: Com `continue`

Observação

O **break** é útil quando uma **regra de negócio**, ao se tornar verdadeira, deve interromper toda a execução. O **continue** é útil quando se quer apenas *desconsiderar* uma determinada iteração, sem parar o laço.

5 Comando while

O laço **while** (“enquanto”) determina que, **enquanto uma condição for válida, o bloco de código será executado**. Ele **testa a condição antes de executar o código**: se a condição já for inválida no primeiro teste, o bloco **nem chega a ser executado**.

5.1 Sintaxe

```
1 while (expressaoBooleana) {  
2     // corpo: comandos executados enquanto a condição for verdadeira  
3 }
```

Listing 7: Estrutura do while

O comando será executado até que a expressão de validação se torne **falsa**.

Observação

É preciso garantir que, em algum momento, a condição se torne **falsa**. Caso contrário, cria-se um **laço infinito** (o bloco nunca para de executar).

5.2 Exemplo 3 — A mesada do Joãozinho

Enunciado do Exemplo

Joãozinho recebeu **R\$ 50,00** de mesada e foi a uma loja de doces para gastar todo o dinheiro. Enquanto o valor da mesada for **maior que zero**, ele adiciona ao carrinho um doce de valor aleatório (entre R\$ 2 e R\$ 8) e desconta esse valor da mesada. Ao final, exiba o valor restante da mesada e uma mensagem informando que Joãozinho gastou toda a sua mesada em doces.

Solução

```
1 import java.util.concurrent.ThreadLocalRandom;
2
3 public class ExemploWhile {
4     public static void main(String[] args) {
5         double mesada = 50.0;
6
7         while (mesada > 0) {
8             double valorDoce = valorAleatorio();
9             if (valorDoce > mesada) {
10                 valorDoce = mesada;    // adaptação: evita mesada
                                     negativa
11             }
12             System.out.println("Doce do valor: " + valorDoce + "
                                adicionado no carrinho");
13             mesada = mesada - valorDoce;
14         }
15
16         System.out.println("Mesada: " + mesada);
17         System.out.println("Joãozinho gastou toda a sua mesada em
                                doces");
18     }
19
20     private static double valorAleatorio() {
21         return ThreadLocalRandom.current().nextDouble(2, 8);
22     }
23 }
```

Listing 8: ExemploWhile.java

Explicação:

- `double mesada = 50.0;` — a mesada inicia com 50.
- `while (mesada > 0)` — enquanto a mesada for maior que zero, o laço executa: pega um doce de valor aleatório e subtrai da mesada.
- `valorAleatorio()` — método auxiliar que retorna um `double` aleatório entre 2 e 8, usando `ThreadLocalRandom`. (Não é requisito do `while`; serve apenas para simular valores aleatórios.)
- `if (valorDoce > mesada) valorDoce = mesada;` — adaptação para que a mesada não fique negativa (regra de negócio). Sem esse trecho o código também funciona, mas poderia terminar com valor negativo.

Observação

A quantidade de iterações não é fixa: depende dos valores aleatórios sorteados. O laço executa enquanto a condição for verdadeira — neste exemplo, até a mesada chegar a zero (ou próximo disso) e a condição `mesada > 0` se tornar falsa.

6 Comando do while

O `do while` (“faça enquanto”) tem a mesma proposta do `while` (executa enquanto a condição for válida), mas com uma diferença fundamental: **testa a condição depois de executar o código**, garantindo que o bloco seja executado **pelo menos uma vez**.

Assim, mesmo que a condição já seja inválida no primeiro teste, o bloco executa uma vez antes de parar.

6.1 Sintaxe

```
1 do {  
2     // corpo: executado ao menos uma vez  
3 } while (expressaoBooleana);
```

Listing 9: Estrutura do `do while`

Note o ponto e vírgula (;) ao final da linha do `while` — obrigatório nessa estrutura.

6.2 Exemplo 4 — O telefone tocando

Enunciado do Exemplo

Joãozinho ligou para um amigo. Enquanto o telefone estiver **tocando**, ele aguarda; quando o amigo **atender**, o fluxo é interrompido e parte-se para a conversa (“Alô!”). Use um `do while` para simular a chamada: o telefone deve “tocar” ao menos uma vez antes de verificar se o amigo atendeu.

Solução

```
1 import java.util.Random;  
2  
3 public class ExemploDoWhile {  
4     public static void main(String[] args) {  
5         System.out.println("Discando...");  
6  
7         do {  
8             System.out.println("Telefone tocando");  
9         } while (tocando());  
10  
11         System.out.println("Alô!!!");  
12     }  
13  
14     private static boolean tocando() {  
15         boolean atendeu = new Random().nextInt(3) + 1 == 1;    //  
16             valor aleatório entre 1 e 3  
17         System.out.println("Atendeu? " + atendeu);  
18         return !atendeu;    // continua "tocando" enquanto NÃO  
19             atendeu  
20     }  
21 }
```

Listing 10: ExemploDoWhile.java

Explicação:

- `do ... while (tocando());` — o corpo executa ao menos uma vez (“Telefone tocando”) e, só depois, a condição é testada.
- `tocando()` — método auxiliar que retorna `boolean` (*sim ou não*: atendeu ou não atendeu). Gera um valor aleatório entre 1 e 3; se for igual a 1, significa que **atendeu**.
- `return !atendeu;` — usa o operador unário de negação (`!`, “exclamação”). Se atendeu, nega para `false` e o laço para de “tocar”; senão, retorna `true` e o telefone continua tocando.

Observação

Por ser aleatório, o número de “toques” muda a cada execução: às vezes o amigo atende de primeira; outras vezes o telefone toca várias vezes. O que é garantido é que o telefone **toca ao menos uma vez** antes da verificação — o que não aconteceria com o `while`, que testa a condição antes.

7 Comparação: `while` vs `do while`

<code>while</code>	<code>do while</code>
Testa a condição antes de executar.	Testa a condição depois de executar.
Pode não executar nenhuma vez (se a condição já for falsa no primeiro teste).	Executa pelo menos uma vez , mesmo que a condição seja falsa de início.

8 Resumo e dicas de prática

- `for` — ideal quando se conhece (ou se controla) a quantidade de repetições por meio de um contador.
- `while` — ideal quando a repetição depende de uma condição que é validada antes de cada execução.
- `do while` — ideal quando o bloco **precisa** executar ao menos uma vez antes da validação.
- `break` interrompe todo o laço; `continue` pula apenas a iteração atual.
- Em *arrays*, o índice começa em 0.

Para consolidar o conteúdo, o professor recomenda praticar de forma isolada, por exemplo:

- um contador de 0 a 100;
- um contador de números positivos;
- definir cenários onde `break` e `continue` se encaixam;
- criar um *array* e listar todas as frutas de uma cesta de frutas;
- usar a depuração (*breakpoint*) para acompanhar o valor das variáveis a cada iteração.

9 Exercícios de fixação

Questões no estilo de prova para consolidar os conceitos de estruturas de repetição, todas com resolução comentada.

9.1 Exercício 1 — while com condição inicial falsa

Exercício

O que será impresso no console pelo código abaixo?

```
1 public static void main(String[] args) {  
2     boolean condicao = false;  
3     while (condicao) {  
4         System.out.println("executou ... ");  
5     }  
6 }
```

Resolução

Nada é impresso (a execução termina sem produzir nenhuma linha no console). O **while** testa a condição **antes** de executar o bloco. Como **condicao** já começa com **false**, a condição é falsa no primeiro teste e o corpo do laço **não é executado nenhuma vez**.

Se a estrutura fosse **do while**, o bloco executaria **uma vez** antes de testar e parar.

9.2 Exercício 2 — length de um array

Exercício

O que será impresso no console pelo código abaixo?

```
1 String[] nomes = {"Camila", "Venilton", "Leonardo", "Renan", "  
    Rafael"};  
2 System.out.print(nomes.length);
```

Resolução

5.

nomes.length retorna a **quantidade de elementos** do array. O array foi criado com 5 nomes, logo **length** vale 5.

- Em *array*, **length** é um **atributo** (sem parênteses): **nomes.length** — e não **nomes.length()**, que é o caso de **String**.
- O índice começa em 0: o último elemento é **nomes[4]**. Acessar **nomes[5]** lançaria **ArrayIndexOutOfBoundsException**.
- **System.out.print** não pula linha (diferente de **println**).

9.3 Exercício 3 — Classificação das estruturas de repetição

Exercício

As estruturas de repetição podem ser classificadas em:

- (a) Repetição com teste no início (**while**), Repetição com teste no final (**do while**), Repetição contada (**for**).
- (b) Repetição com teste no início (**for**), Repetição com teste no final (**do while**), Repetição contada (**while**).
- (c) Repetição com teste no início (**do while**), Repetição com teste no final (**while**), Repetição contada (**for**).
- (d) Repetição com teste no início (**for**), Repetição com teste no final (**do while**), Repetição contada (**while**).
- (e) Repetição com teste no início (**do while**), Repetição com teste no final (**for**), Repetição contada (**while**).

Resolução

Alternativa (a).

Estrutura	Classificação	Motivo
while	Teste no início	verifica a condição <i>antes</i> de executar; pode não rodar nenhuma vez
do while	Teste no final	executa ao menos uma vez e só <i>depois</i> testa
for	Repetição contada	usa variável contadora (inicialização + condição + incremento)

Observação: as alternativas (b) e (d) são idênticas entre si (ambas incorretas).

9.4 Exercício 4 — for como acumulador

Exercício

O que será impresso no console pelo código abaixo?

```
1 public static void main(String[] args) {  
2     int numero = 1;  
3     for (int x = 1; x < 2; x++) {  
4         numero = numero + x;  
5     }  
6     System.out.println("O valor de número é: " + numero);  
7 }
```

Resolução

O valor de número é: 2.

O corpo do laço executa **uma única vez** (com $x = 1$), porque na verificação seguinte $2 < 2$ já é falso.

Passo	x	numero
início	—	1
$x = 1$; teste $1 < 2 \rightarrow$ verdadeiro	1	1
$numero = 1 + 1$	1	2
$x++ \rightarrow 2$; teste $2 < 2 \rightarrow$ falso	2	2

numero é um **acumulador**: começa em 1 e recebe o valor de x a cada iteração.

9.5 Exercício 5 — Loop infinito no do while

Exercício

O que será impresso no console pelo código abaixo?

```
1 int num = 5, count = 1;
2 do {
3     num += count;
4     System.out.println(num);
5 } while (count <= 3);
```

Resolução

Loop infinito: o programa imprime 6, 7, 8, 9, 10, ... sem parar.

A variável count **nunca é alterada** dentro do laço (permanece 1), então a condição $count \leq 3$ é sempre verdadeira.

Iteração	count	num += count	imprime
1ª	1	$5 + 1$	6
2ª	1	$6 + 1$	7
3ª	1	$7 + 1$	8
...	1	...	...

Correção — incrementar count dentro do laço para que a condição eventualmente se torne falsa:

```
1 int num = 5, count = 1;
2 do {
3     num += count;
4     System.out.println(num);    // imprime 6, 7, 8, 9
5     count++;
6 } while (count <= 3);
```

Observação

Em todo `while`/`do while`, a variável usada na condição precisa ser alterada dentro do laço — caso contrário, cria-se um loop infinito.